

1 Task Overview

Modern SSDs include built-in security features such as encryption, authentication, and access control. SSDs that provide such protection are often called Self-Encrypting Drives (SEDs). These features are controlled by SSD security protocol commands.

In this project, your goal is to build a model that checks whether an SSD responds correctly to such commands.

- The input is a command-response log collected from an SSD.
- The model inspects the whole log and judges the final response.
- The output is a test verdict: `PASS` or `FAIL`.

Concretely, your program should read a trajectory from the provided dataset and output one label, `PASS` or `FAIL`, for the final target command.

The key difficulty is that the SSD is stateful. The same command may be correct or incorrect depending on what happened earlier: whether a session was opened, which authority was authenticated, and whether a protected storage region is locked or unlocked. Therefore, the model must infer the current SSD state from the previous command-response history before judging the final response.

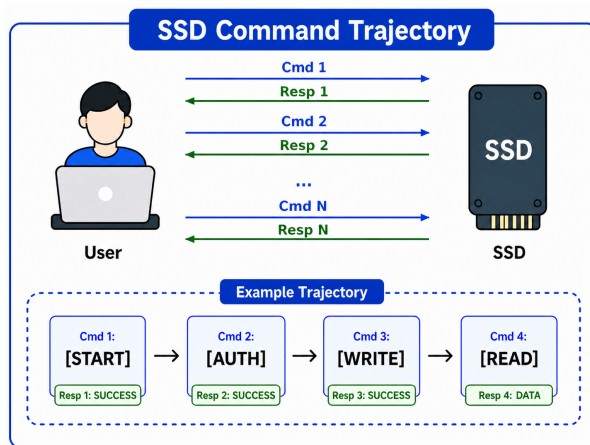


Figure 1: Example SSD command-response trajectory

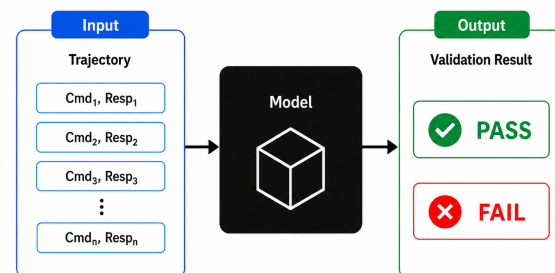


Figure 2: PASS/FAIL prediction from a trajectory

Figure 1 shows an SSD command trajectory: the host sends a command, the SSD returns a response, and this process repeats. Figure 2 summarizes the learning task: the model receives the full trajectory and predicts whether the final response is correct according to the protocol.

1.1 Key Concepts for This Task

1.2 What a Command-Response Record Looks Like

In the dataset, each trajectory consists of command-response records. A command is what the host sends to the SSD, and a response is what the SSD returns for that command. For TCG security protocol commands, this interaction is represented as an `IF-SEND / IF-RECV` pair.

Term Project

For example, the host may send the following command to start a session:

```
[Command #55] IF-SEND
Method: StartSession
Invoking ID: 00..00 FF
Args:
SPID: 0000020500000002
Write: 1
```

The SSD may then return the following response:

```
[Command #55] IF-RECV
Response:
HostSessionID: 00000000
SPSessionID: 00001234
Status: SUCCESS
```

This means that the host requested a `StartSession` operation, and the SSD accepted it by returning `SUCCESS` along with session identifiers. Later commands in the trajectory may depend on these session identifiers and on the state created by this successful response.

A *test verdict* is the tester's judgment of whether that response is allowed by the protocol. The verdict is `PASS` if the response is correct under the current protocol state, and `FAIL` if it violates the specification.

Therefore, `PASS` does not mean that the SSD returned `SUCCESS`. It means that the SSD returned the **correct response**. Similarly, `FAIL` does not mean that the SSD returned an error. It means that the SSD returned a response that was **not allowed** by the protocol.

Examples. Both examples have the same target command and the same device response: `Set(...)` returns `SUCCESS`. The verdict changes only because the previous authentication state is different.

✓ PASS Authenticated access

Previous context

```
StartSession(LockingSP, Write=0)
→ SUCCESS
Authenticate(Admin, Password)
→ SUCCESS
```

Target command and device response

```
Set(Locking::GlobalRange,
WriteLocked=True)
→ SUCCESS
```

- Session was opened.
- Host was **authenticated**.
- The protected object was modified with proper authority.

Verdict: **PASS**

✗ FAIL Unauthenticated access

Previous context

```
StartSession(LockingSP, Write=0)
→ SUCCESS
// Authenticate not performed
```

Target command and device response

```
Set(Locking::GlobalRange,
WriteLocked=True)
→ SUCCESS
```

- Session was opened.
- Host was **not authenticated**.
- Since this is a protected operation, the SSD should have returned **NOT AUTHORIZED**.

Verdict: **FAIL**

Stateful reasoning. The final command usually cannot be judged in isolation. The model should infer the current SSD state from the previous command-response pairs.

- **Session state:** Is a valid session currently open?
- **Authentication state:** Has the host authenticated as the required authority?

- **Locking state:** Is the target storage region enabled, locked, or unlocked?

Thus, an error response can be correct, and a `SUCCESS` response can still be a specification violation. In other words, the model should behave like a test oracle: infer the state, determine the protocol-required response, and compare it with the actual device response.

Project objective. Each team will design a verifier for SSD protocol compliance.

- The verifier receives command trajectories and predicts `PASS` or `FAIL` for the final target command.
- You may use deep learning models, LLMs, prompting, retrieval, fine-tuning, rule-based components, or other modeling strategies.
- The main objective is stateful protocol reasoning, not simple pattern matching.

2 Formal Task Definition

A test case is given as a sequence of JSON command-response records:

$$X = ((c_1, r_1), (c_2, r_2), \dots, (c_N, r_N)),$$

where each (c_t, r_t) corresponds to a single command execution log. The command c_t is sent by the host, and r_t is the response returned by the SSD.

The final pair (c_N, r_N) is the target command and its actual response. Only the final response r_N is evaluated. The previous pairs $(c_1, r_1), \dots, (c_{N-1}, r_{N-1})$ are context for inferring the SSD state.

Given X , the goal is to predict:

$$y \in \{\text{PASS}, \text{FAIL}\}.$$

- `PASS`: the final response is compliant with the specification under the inferred state.
- `FAIL`: the final response violates the specification.

Each team will design a verifier for this binary prediction task. You may use deep learning models, LLMs, prompting, retrieval, fine-tuning, rule-based components, or other modeling strategies.

3 Background: TCG Storage and Test Cases

This section explains how to read the provided test cases and how to connect them to the TCG Storage specifications. The goal is not to memorize every table in the documents. Instead, focus on identifying the final target command, tracking the protocol state created by previous commands, and checking whether the final response is compliant.

3.1 Specification Documents at a Glance

The project uses several specification and reference documents.

- **TCG Storage Architecture Core Specification.** This is the base protocol document. It defines the common architecture and vocabulary: Host, TPer, Security Provider (SP), sessions, methods, tables, objects, UIDs, authorities, access control, status codes, and method-call syntax.

Term Project

- **TCG Storage Opal SSC Specification.** This specializes the Core Specification for Opal self-encrypting drives. It defines Opal-specific behavior, including Admin SP, Locking SP, default authorities, default credentials, locking ranges, media encryption keys, and required Opal features.
- **Opal SSC Application Note.** This is a workflow-oriented document. It is useful for understanding common scenarios such as taking ownership, activating the Locking SP, configuring locking ranges, invoking GenKey, and reverting the drive.
- **README and dataset files.** These describe the exact JSON format, input/output interface, file layout, and submission procedure used in this project.

3.2 How to Read a TCG Method Call

A TCG command in the dataset is usually represented as a method call. In the Core Specification, a method call is written conceptually as:

```
<InvokingID>.<MethodName>[
  Required Parameter(s),
  Optional Parameter(s)
]
=>
[ Result ]
```

For this project, you can interpret the fields as follows.

Field	Meaning
invoking_id	The object, table, SP, or session-manager object on which the method is invoked. In other words, it tells you <i>what the command is operating on</i> .
method.name	The method being called, such as StartSession, Get, Set, Activate, or GenKey.
method.args.required	Required parameters. These are positional parameters defined by the method signature.
method.args.optional	Optional parameters. In the Core Specification, optional parameters are encoded as named values.
output.status.codes	The SSD's actual method-level status, such as Success, NOT_AUTHORIZED, or INVALID_PARAMETER.
output.return.values	Values returned by the SSD, such as session identifiers, table values, or other method results.

Reference cue from the Core Specification. The Core Specification explains that a method header consists of an InvokingID and a MethodID: the InvokingID identifies the table, object, or SP on which the method operates, while the MethodID identifies the method being invoked. Optional parameters are submitted after required parameters as named value pairs.

Example. The following JSON record invokes Set on a C_PIN object.

```
{
  "input": {
    "method": {
      "name": "Set",
```

```

    "args": {
      "required": {},
      "optional": {
        "Values": [
          {"0x03": "D0104E65775F5349445F50617373776F7264"}
        ]
      }
    },
    "invoking_id": {
      "uid": "00 00 00 0B 00 00 00 01",
      "name": "C_PIN"
    }
  },
  "output": {
    "return_values": [],
    "status_codes": "Success"
  }
}

```

This should be read as: call `Set` on a `C_PIN` object, write a new value to column `0x03`, and observe that the SSD returned `Success`.

3.3 Reading Example Test Cases

A test case is a trajectory: a sequence of command-response records. The **last record** is the target command-response pair. The earlier records are context for inferring the SSD state.

Example 1: TC-3 — Updating the SID Credential

File: `tc3.json`

TC-3 is a compact credential-update flow. The host opens an Admin SP session, reads the default `MSID`-related credential value, opens an authenticated `SID` session, updates the `SID` PIN, and then tries to open another `SID` session using the new PIN.

Here, `SID` is the owner authority of the drive, and `C_PIN` denotes a credential object that stores PIN-related values. This example tests whether the updated credential takes effect in a later session.

Step	Command	Actual response
1	<code>StartSession(AdminSP, Anybody, Write=1)</code>	SUCCESS
2	<code>Get(C_PIN_MSID, col=3)</code>	SUCCESS, returns credential value
3	<code>EndSession()</code>	SUCCESS
4	<code>StartSession(AdminSP, SID, HostChallenge=[old PIN], Write=1)</code>	SUCCESS
5	<code>Set(C_PIN_SID, Values=[new PIN])</code>	SUCCESS
6	<code>EndSession()</code>	SUCCESS
7	<code>StartSession(AdminSP, SID, HostChallenge=[new PIN], Write=1)</code>	SUCCESS

How to interpret it. The final target command is Step 7: `StartSession` as `SID` using the new PIN. To judge whether the final `SUCCESS` is correct, the verifier must infer the prior state.

- Step 1 opens an Admin SP session.
- Step 2 reads a credential value from the `C_PIN` table.

Term Project

- Step 4 opens an authenticated `SID` session.
- Step 5 updates the `SID` credential to a new PIN.
- Therefore, Step 7 should succeed when the host authenticates as `SID` using the new PIN.

Relevant specification cue. For this case, the important concepts are session startup, authority authentication, the `CLPIN` credential object, and how credential updates affect later authentication attempts.

Expected verdict: **PASS**. The final `SUCCESS` is compliant because the `SID` credential was successfully updated before the final session attempt.

Example 2: TC-20 — GenKey Failure Case

File: `tc20.json`

TC-20 tests whether key regeneration changes the media encryption key used for a locking range. The trajectory first performs setup operations such as taking ownership, activating the `LockingSP`, configuring a locking range, and preparing the associated media key. It then writes a known pattern, reads it back, invokes `GenKey`, and finally reads the same LBA range again.

An SED stores user data encrypted with a media encryption key. If this key is regenerated, old ciphertext should no longer decrypt to the same plaintext pattern. Therefore, after a successful `GenKey`, reading the same LBA range should not return the old pattern. This is the basic idea behind cryptographic erase.

Step	Command	Actual response
1–32	Take ownership, activate <code>LockingSP</code> , configure the locking range, and prepare the media key	<code>SUCCESS</code>
33	<code>Write(LBA=80--87, pattern=0x8E)</code>	<code>pass</code>
34	<code>Read(LBA=80--87)</code>	<code>Pattern 8E</code>
35–36	Open an authenticated <code>Admin1</code> session and get the media-key object	<code>SUCCESS</code>
37	<code>GenKey(K_AES_256_Range_Key)</code>	<code>SUCCESS</code>
38	<code>EndSession()</code>	<code>SUCCESS</code>
39	<code>Read(LBA=80--87)</code>	<code>8E</code>

The key operation in the JSON looks like this:

```
{
  "index": 37,
  "input": {
    "method": {
      "name": "GenKey",
      "uid": "00 00 00 06 00 00 00 10",
      "args": {
        "required": {},
        "optional": {}
      }
    },
    "invoking_id": {
      "uid": "00 00 08 06 00 03 00 01",
      "name": "K_AES_256"
    },
    "status_codes": "SUCCESS"
  },
  "output": {
    "return_values": [],
    "status_codes": "SUCCESS"
  }
}
```

Term Project

The final target record is the read after GenKey:

```
{
  "index": 39,
  "input": {
    "command": "Read",
    "args": {
      "LBA": "80 ~ 87"
    }
  },
  "output": {
    "command": "Read",
    "args": {
      "result": "8E"
    }
  }
}
```

How to interpret it. The final target command is not a TCG method. It is a normal data Read. However, its correctness depends on the previous TCG method GenKey.

- Step 33 writes Pattern 8E to LBA range 80--87.
- Step 34 confirms that the same pattern can be read back before key regeneration.
- Step 37 invokes GenKey on the K_AES_256 media key object, and the method returns SUCCESS.
- Therefore, the verifier should treat the media key regeneration as having occurred.
- After key regeneration, the old ciphertext should no longer decrypt to the original 8E pattern.

Relevant specification cue. The Core Specification describes GenKey as an object method used for key creation. It fills an existing credential/key object with new key material and returns an empty result list; success or failure is determined by the method status code. The Core Specification also defines the K_AES_256 table as storing AES-256 media encryption keys and associated metadata.

Expected verdict: FAIL. The device reported SUCCESS for GenKey, but the old pattern was still readable. This suggests that the media key was not actually regenerated, so the final response is not compliant.

3.4 Practical Remarks

- **Always identify the final target record.** The label judges only the final response. Earlier records are context.
- **Track state only after successful operations.** If a command failed, do not assume that its intended state change happened.
- **Use symbolic names.** The JSON contains UIDs and byte strings, but the important meaning is often symbolic: AdminSP, LockingSP, SID, Admin1, C_PIN, Locking, or K_AES_256.
- **TCG commands can affect later non-TCG data commands.** For example, GenKey can change the expected result of a later Read, even though the final command is not itself a TCG method.
- **Consult the specification selectively.** For each test case, first identify the method, invoking object, authority, and state variable. Then look up only the relevant Core, Opal SSC, or Application Note sections.

3.5 Final Reminder: Verdict vs. Device Response

A device response is what the SSD actually returned: a status code, returned values, session identifiers, or data-read results.

A test verdict asks a different question:

Was the **final response** allowed by the specification under the state created by the previous trajectory?

Therefore:

- SUCCESS can be PASS if success is allowed.
- SUCCESS can be FAIL if the command should have been rejected or should have produced a different effect.
- An error response can be PASS if the error is the correct protocol behavior.
- An error response can be FAIL if the command should have succeeded.

4 Evaluation Criteria

- Final evaluation will be conducted on a private dataset, which includes test scenarios not present in the public dataset or reflected in the leaderboard. Therefore, you should not perform black-box optimization or attempt to reverse-engineer the leaderboard score.
- The final score will be measured by accuracy, defined as the fraction of test cases for which the submitted prediction matches the ground-truth label:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\hat{y}_i = y_i\},$$

where N is the number of test cases, $\hat{y}_i$ is the predicted label for the i -th test case, y_i is the ground-truth label, and $\mathbf{1}\{\cdot\}$ denotes the indicator function.

- Additionally, each team is required to submit a 4-page report that describes your method and analysis.

5 Practice Server Usage

In this term project, we provide a practice server for each team. Each practice server has a single NVIDIA L40S GPU with 48GB of GPU memory, 24 CPU cores, 60GB of CPU memory, and separate storage volumes for the user home directory and workspace. Each team is assigned a unique IP address and port number for SSH access. The server access information has been announced by email. Here are basic instructions about the server.

- Contact TA if you encounter any login issues.
- The base system storage is limited. However, two persistent storage volumes are provided for your project:
 - /home/student: 100GB

Term Project

- /workspace: 1TB

- **Important: Files stored under /home/student and /workspace will be preserved even if the server encounters system issues.** Files stored outside these directories may not be recoverable.
- The public dataset is stored in the /dl2026/dataset directory on each server.
- You may use /home/student for personal configuration files, source code, scripts, and small project files. We recommend using /workspace for datasets, model checkpoints, generated outputs, and other large files.
- To submit your solution, run:

```
submit
```

at your project root directory. By default, this command submits the current directory. You may also run:

```
submit -n, --job-name <name>
submit -d, --dir <your_directory>
submit --list
```

- The submitted directory must include:
 - src/: your solver code,
 - setup.sh: a script for preparing the evaluation environment,
 - pyproject.toml and uv.lock: dependency information.

Optionally, you may include artifacts/ for additional files such as fine-tuned model weights, checkpoints, or generated resources.

- **Note:** The evaluation server will use a separate evaluator and dataset. Do not assume that files such as evaluate.py or predictions.jsonl in your local directory will be submitted or used for final grading.
- You may test your solution locally before submitting it:

```
cd /workspace/project
bash setup.sh
python evaluate.py
```

6 Submission Policy

- You can make regular submissions using the submit command. The number of submissions per day is limited by the server policy. You can check your submission history and current status by running:

Term Project

```
submit --list
```

- Each team can have at most one queued or running job at a time. If a previous job is still queued or running, a new submission will be rejected.
- The evaluation process consists of two phases:
 - **Setup phase:** runs `setup.sh` and prepares the environment. Network access is available in this phase. The time limit is 20 minutes.
 - **Evaluation phase:** runs the evaluator on the dataset. Network access is disabled in this phase. The time limit is 3 hours.

If either phase exceeds its time limit, the submission will receive zero score.

- The submission script packages only the allowed project files, such as `src/`, `setup.sh`, `pyproject.toml`, `uv.lock`, and optionally `artifacts/`.
- The submission archive may contain large files such as model checkpoints. However, the archive size must not exceed 12GB. Submissions larger than 12GB will be rejected.
- If you use additional model weights or data files during evaluation, they must be prepared in `setup.sh` or included in `artifacts/`. The evaluation phase will run without network access.
- Because the evaluation phase runs without network access, only the following pretrained models will be available by default on the evaluation server. These models will be pre-downloaded into the shared Hugging Face cache by the course staff:
 - `openai/gpt-oss-20b`
 - `Qwen/Qwen3.5-{0.8B, 2B, 4B, 9B}`
 - `Qwen/Qwen3.5-27B-FP8`
 - `Qwen/Qwen3.5-35B-A3B-FP8`
 - `google/gemma-4-26B-A4B-it`
 - `google/gemma-4-{E2B, E4B, 31B}-it`

Other pretrained models will not be available during evaluation unless they are explicitly included in your submission or downloaded during `setup.sh`.

- The submission deadline will be announced separately.
- If you try to attack the submission server, the evaluation server, or another team's server, your team will receive a strong penalty.